

ЛАБОРАТОРНАЯ РАБОТА № 1

ОСНОВЫ КРИПТОСИСТЕМЫ RSA

Цель работы – изучение криптосистемы RSA, реализация алгоритмов зашифрования и расшифрования сообщений, формирования и проверки электронной цифровой подписи.

Теоретические сведения

1. Краткое описание криптосистемы RSA

Криптосистема RSA, предложенная в 1977 году Р. Ривестом (Ron Rivest), А. Шамиром (Adi Shamir) и Л. Адлеманом (Leonard Adleman) и названная по первым буквам фамилий авторов, широко используется для шифрования с открытым ключом и электронной (цифровой) подписи. Криптосистема RSA является составной частью многих стандартов, а также используется во многих приложениях и протоколах, например, PGP, SSL/TLS, SSH и др.

Принцип действия криптосистемы заключается в следующем. Пусть p и q – два больших простых числа и пусть $n = pq$. Выбирается число e , $1 < e < \varphi(n) - 1$, взаимно простое со значением функции Эйлера $\varphi(n) = (p - 1)(q - 1)$, и вычисляется d – мультипликативно обратное к e : $d \equiv e^{-1}(\text{mod } \varphi(n))$. Числа e и d называются открытым и закрытым показателями соответственно. Открытым ключом является пара (n, e) , закрытым ключом – пара (n, d) . Множители p и q известны только владельцу секретного показателя d (или могут быть вычислены с помощью этого показателя).

Для зашифрования сообщения m нужно вычислить шифртекст $c \equiv m^e (\text{mod } n)$, а для расшифрования – возвести шифртекст в степень d : $m \equiv c^d (\text{mod } n)$.

2. Применение криптосистемы RSA для шифрования

Одна операция зашифрования в системе RSA позволяет зашифровать сообщение, представленное числом m , где $0 \leq m < n$. Текст более длинных сообщений необходимо разбивать на блоки. То же

справедливо и для файлов: преобразование из потока байтов в число для зашифрования подразумевает разделение файла на блоки байтов, преобразование каждого такого блока в число и зашифрование каждого блока в отдельности.

При преобразовании сообщения (последовательности байтов) в число можно использовать два варианта представления. В первом варианте полагается, что первые байты сообщения определяют старшие цифры числа (Most Significant Bytes first, MSB), а во втором – первые байты сообщения являются младшими цифрами числа (Least Significant Bytes first, LSB).

Как и все несимметричные алгоритмы, алгоритм шифрования RSA требует значительных вычислительных ресурсов для выполнения каждой операции зашифрования. Поэтому на практике несимметричные алгоритмы шифрования часто используют совместно с симметричными алгоритмами.

Пусть используется симметричный блочный шифр C с размером блока l_b и длиной ключа l_k . Для передачи каждого нового сообщения m генерируется случайный ключ k шифра C . Сгенерированный ключ представляется в виде числа и шифруется алгоритмом RSA. Основное сообщение разбивается на блоки длиной l_b и шифруется на ключе k симметричным шифром C .

Получателю такого составного сообщения передаются зашифрованный при помощи RSA ключ симметричного алгоритма $c = RSA_e(k)$ и шифртекст $C_k(m)$ основного сообщения $C_k(m)$. Получатель с помощью закрытого ключа d расшифровывает ключ k симметричного алгоритма, а затем основное сообщение m .

Размер ключа l_k обычно значительно меньше длины числа n в байтах, поэтому для шифрования такого ключа требуется выполнение только одной операции RSA.

Применение такого подхода, помимо значительного увеличения скорости передачи длинных сообщений, обладает еще одним преимуществом: упрощается передача широковещательных сообщений. Для нескольких получателей ключ, на котором зашифровано сообщение m , передается зашифрованным на различных ключах зашифрования: $RSA_{e_1}(k)$, $RSA_{e_2}(k)$, ..., $RSA_{e_n}(k)$. Получатели такого сообщения,

каждый на своем закрытом ключе, расшифровывают промежуточный ключ k и получают доступ к сообщению.

Вместе с зашифрованным промежуточным ключом необходимо передавать также значение или идентификатор ключа шифрования RSA, чтобы каждый из получателей смог определить, какой именно блок из множества переданных необходимо расшифровать для получения промежуточного ключа.

3. Применение криптосистемы RSA для подписи

До появления криптографии с открытым ключом целостность и подлинность документов в условиях взаимного недоверия контролировались, как правило, по материальным признакам. Документы имели конкретное физическое воплощение, например, были напечатаны на гербовой бумаге или заверены печатью, но с ростом информатизации стали терять уникальные физические качества.

Использование симметричных криптографических алгоритмов не позволяет установить подлинность и целостность электронного документа в условиях взаимного недоверия. Например, если получатель сообщения утверждает, что получил от отправителя некоторое заверенное сообщение, а отправитель это отрицает, то один из них лжет. Установить, кто именно говорит неправду, затруднительно, так как оба они обладают равными возможностями в части формирования имитозащищенных сообщений. Осуществить аутентификацию данных и их источника в этом случае позволяет электронная подпись.

Электронная подпись в цифровых документах играет ту же роль, что и подпись, поставленная от руки в бумажных документах: это данные, присоединяемые к передаваемому сообщению и подтверждающие, что автор подписи (отправитель) составил или заверил данное сообщение. Получатель сообщения или третья сторона с помощью электронной подписи может проверить, что автором сообщения является именно владелец подписи и что в процессе передачи не была нарушена целостность данных.

Протокол электронной подписи предполагает два этапа: формирование и проверку. Часто способы формирования и проверки электронной подписи допускают некоторую свободу действий, которую

нет смысла строго регламентировать. Это позволяет говорить о схеме подписи.

Таким образом, при разработке протокола подписи нужно:

- создать пару сообщение/подпись так, чтобы ее невозможно было подделать;
- уметь проверить, что подпись действительно принадлежит указанному владельцу.

Кроме того, подпись должна быть построена так, чтобы отправитель, подписавший сообщение, не смог затем отрицать перед получателем или третьей стороной факт подписания, утверждая, что подпись подделана.

Обычно, говоря о схеме электронной подписи, имеют в виду следующую ситуацию:

- отправитель знает содержание сообщения, которое он подписывает;
- получатель, зная открытый ключ проверки подписи, может проверить правильность подписи в любое время без какого-либо разрешения или участия отправителя;
- безопасность схемы подписи (то есть трудность подделки, невозможность отказа от подписи и т. д.) обеспечивается положениями теории сложности.

При формировании электронной подписи по классической схеме отправитель сообщения сначала применяет к исходному тексту хэш-функцию, а затем вычисляет подпись с помощью закрытого ключа формирования подписи.

Получатель сообщения выполняет проверку подписи: сначала вычисляет хэш-образ полученного сообщения с помощью той же самой хэш-функции, которую использовал отправитель, а затем с помощью открытого ключа проверяет, соответствует ли этот хэш-образ полученной электронной подписи.

4. Схема подписи RSA

Протокол 1.1. Схема подписи RSA

Вход отправителя. Составное число n , закрытый показатель d .

Вход получателя. Составное число n , открытый показатель e .

Результат. Формирование и проверка подписи.

Для формирования подписи для сообщения m отправитель выполняет следующие действия.

1. Вычисляет хэш-образ $r \leftarrow h(m)$ сообщения m .
2. Зашифровывает число r на своем закрытом ключе d , то есть вычисляет экспоненту $s \leftarrow r^d \pmod{n}$.

Подписью для сообщения m является число s .

Для проверки подписи получатель выполняет следующие действия.

1. Расшифровывает подпись s на открытом ключе e отправителя, то есть вычисляет $t \leftarrow s^e \pmod{n}$, восстанавливая предполагаемый хэш-образ t сообщения m .
2. Вычисляет хэш-образ $r \leftarrow h(m)$ сообщения m .
3. Проверяет равенство $r = t$. Если оно выполняется, то подпись принимается, в противном случае – подпись неверна. ■

Схема подписи RSA может быть рассмотрена как категория, объектами которой являются пары сообщение/подпись, а морфизмами – отображения таких пар, удовлетворяющие проверочным соотношениям. Если (m, s) – подлинное подписанное сообщение, то пара (m', s') тоже будет правильно подписанным сообщением, если $h(m') \equiv h^k(m) \pmod{n}$ и $s' \equiv s^k \pmod{n}$ для некоторого целого k . Новая пара (m', s') может быть создана следующим алгоритмом: выбрать сообщение m' , вычислить $h(m')$, найти логарифм $h(m')$ по основанию $h(m)$ и вычислить s' . Можно наоборот, сначала найти $r' \equiv h^k(m) \pmod{n}$ для известного k , а затем подобрать m' так, чтобы выполнялось равенство $h(m') = r'$. Поэтому стойкость подписи RSA не может превышать сложности вычисления логарифма в группе $(\mathbb{Z}/n\mathbb{Z})^*$ или сложности обращения хэш-функции.

Если нарушитель составляет текст для подписи, то он может подготовить коллизию (m, m') такую, что $h(m) = h(m')$, дать на подпись сообщение m , а затем подменить подписанное сообщение m сообщением m' . В этом случае стойкость схемы подписи не может превышать сложности вычисления коллизий хэш-функции. Приведенные рассуждения показывают, что необходимо согласовывать сложности

разложения числа n на множители, логарифмирования и нахождения коллизий.

5. Представление ключей RSA

Открытый и закрытый ключи RSA обладают определенной структурой, например, нельзя поменять местами модуль и экспоненту. Кроме того, длины компонент могут сильно различаться. Так, длины ключей RSA в различных системах могут отличаться в разы. Стойкость криптосистемы со временем падает, и для компенсации необходимо постоянно увеличивать длину ключей. Поэтому к методу представления ключа как массива байтов предъявляется ряд требований, а именно:

- информация должна храниться в определенном известном формате;
- не должно быть ограничений по максимальной возможной длине компонент структуры;
- не должно быть выравнивания остальных компонент по наиболее длинной.

Этим требованиям удовлетворяет представление ключа в виде ASN.1-структуры. Такое представление предназначено для хранения и передачи структурированных данных. Подробное описание типов данных в ASN.1-структуре, форматов зашифрованного файла и файла подписи представлено в *Приложении А*.

Контрольные вопросы

1. Какие задачи положены в основу безопасности системы RSA?
2. Показать, что схема RSA работает корректно для любого сообщения $m \in \mathbb{Z}/n\mathbb{Z}$.
3. Доказать, что задача разложения числа n на множители и задача вычисления функции Эйлера $\varphi(n)$ полиномиально эквивалентны.
4. Показать, что схема RSA обладает свойством гомоморфности относительно операции умножения.

Порядок выполнения работы

Получить у преподавателя вариант задания и разработать программу **П-1**, которая:

а) выполняет зашифрование и расшифрование файла с использованием алгоритма RSA. В режиме зашифрования программа должна принимать на вход файл сообщения (произвольного формата), открытый ключ RSA, ключ симметричного алгоритма; на выходе – возвращать зашифрованный файл, заголовок которого соответствует нотации ASN.1, описанной в *Приложении А*. Зашифрование файла необходимо производить с использованием вспомогательного симметричного алгоритма AES-256 в режиме CBC, реализация симметричного алгоритма не требуется. Ключ шифрования симметричного алгоритма – случайный или задан в программе. Случайный ключ шифрования алгоритма AES (32 байта) необходимо представить как число для шифрования алгоритмом RSA, порядок байтов – MSB. Старшие неиспользуемые цифры (байты) числа следует считать нулевыми. В режиме расшифрования программа **П-1** должна принимать на вход зашифрованный файл, закрытый ключ RSA; на выходе – возвращать расшифрованное сообщение;

б) выполняет формирование и проверку электронной подписи с использованием алгоритма RSA. При реализации алгоритма электронной подписи рекомендуется использовать хэш-алгоритм SHA-256. Поскольку хэш-образ, вычисленный по алгоритму SHA-256, может быть длиннее модуля RSA, при проверке подписи следует сравнивать значение $h(m)(\text{mod } n)$ с подписью s сообщения m . В режиме формирования подписи программа **П-1** должна принимать на вход файл, который необходимо подписать, ключ подписи RSA; на выходе – возвращать отдельный файл подписи. В режиме проверки подписи программа должна принимать на вход сообщение и файл подписи; на выходе – возвращать результат «Подпись принимается» или «Подпись неверна».

Содержание отчета

1. Формулировка задания.

2. Выполненная работа:

- а) использованные параметры криптосистемы RSA;
- б) сгенерированный ключ для алгоритма AES-256;
- в) содержание зашифрованного файла в шестнадцатеричном виде (при значительном размере файла – первые 80–100 байт);
- г) файл с подписью RSA.

3. Ответы на контрольные вопросы.

4. Выводы по работе, где сделать предположения о возможных уязвимостях разработанного программного обеспечения и о путях их устранения.

5. Листинг программы.

ЛАБОРАТОРНАЯ РАБОТА № 2

АТАКИ НА КРИПТОСИСТЕМУ RSA, ИСПОЛЬЗУЮЩИЕ ДАННЫЕ О ПОКАЗАТЕЛЯХ

Цель работы – изучение уязвимостей криптосистемы RSA по отношению к атакам, использующим показатели, обладающие определенными свойствами, генерация параметров криптосистемы RSA.

Теоретические сведения

Как показывает практика, правильный выбор множителей p и q не всегда гарантирует стойкость криптосистемы. Существует множество атак, нарушающих безопасность криптосистемы RSA без разложения числа n на множители. Возможность подобных атак, как показано в работе Д. Боне «Twenty years of attacks on the RSA cryptosystem», обусловлена использованием общего модуля n несколькими пользователями, малых значений открытого и закрытого показателей и т.п. Наличие таких атак накладывает значительные ограничения на выбор параметров криптосистем на основе алгоритма RSA.

1. Случай общего модуля

Если в системе RSA у пользователей A и B общее составное число n , но различные открытые e_A, e_B и закрытые d_A, d_B показатели, где $e_A d_A \equiv e_B d_B \equiv 1 \pmod{\varphi(n)}$, то каждый пользователь может разложить число n на множители и тем самым узнать закрытый ключ другого пользователя. Для этого нарушителю (например, пользователю B) достаточно найти квадратный корень из единицы, отличный от ± 1 . Если $t^2 \equiv 1 \pmod{n}$, $t \not\equiv \pm 1 \pmod{n}$, то выполняется сравнение $t^2 - 1 = (t + 1)(t - 1) \equiv 0 \pmod{n}$ и числа $t + 1, t - 1$ имеют нетривиальные общие делители с n . Разложение можно выполнить по аналогии с псевдопростым тестом Миллера–Рабина следующим алгоритмом.

Алгоритм 2.1. Разложение составного числа на множители по известным показателям RSA

Вход. Число n , показатели e_B, d_B такие, что $e_B d_B \equiv 1 \pmod{\varphi(n)}$, открытый показатель e_A другого пользователя.

Выход. Делители p и q числа n , закрытый показатель d_A другого пользователя.

1. Представить разность $e_B d_B - 1$ в виде $2^f s$, где s – нечетное число.
2. Выбрать случайное $a \in \mathbb{Z}/n\mathbb{Z}$ и вычислить $b \leftarrow a^s \pmod{n}$.
3. Вычислять $b^{2^0} \equiv b \pmod{n}$, $b^{2^1} \equiv (b^{2^0})^2 \pmod{n}$, $b^{2^2} \equiv (b^{2^1})^2 \pmod{n}$, ... до получения l такого, что $b^{2^l} \equiv 1 \pmod{n}$. Если $b^{2^{l-1}} \equiv -1 \pmod{n}$, то вернуться на шаг 2, иначе положить $t \leftarrow b^{2^{l-1}} \pmod{n}$.
4. Положить $p \leftarrow \text{НОД}(t + 1, n)$, $q \leftarrow \text{НОД}(t - 1, n)$.
5. Вычислить $\varphi(n) = (p - 1)(q - 1)$ и найти закрытый ключ другого пользователя: $d_A \equiv e_A^{-1} \pmod{\varphi(n)}$.
6. Результат: $(p, q), d_A$. ■

2. Случай малого закрытого показателя

Атака Винера на схему RSA основана на предположении о том, что значение закрытого показателя d достаточно мало. Пусть $n = pq$, $q < p < 2q$, (n, e) – открытый ключ, (n, d) – закрытый ключ схемы RSA. Тогда если $d < \frac{1}{3} \sqrt[4]{n}$, то значение закрытого показателя d может быть вычислено за полиномиальное от n время.

Для доказательства этого предположения используется математический аппарат непрерывных дробей, который позволяет найти приближение к некоторому вещественному числу α с помощью подходящих дробей. Пусть $\frac{P}{Q}$ – несократимая рациональная дробь, и выполняется условие: $\left| \alpha - \frac{P}{Q} \right| < \frac{1}{2Q^2}$, тогда $\frac{P}{Q}$ представляет собой подходящую дробь к числу α .

Открытый и закрытый показатели связаны соотношением $ed \equiv 1 \pmod{\varphi(n)}$, т.е. существует такое $k \in \mathbb{Z}$, что $ed = 1 + k\varphi(n)$, или

$|ed - k\varphi(n)| = 1$. При делении последнего равенства на $d\varphi(n)$ получаем:

$$\left| \frac{e}{\varphi(n)} - \frac{k}{d} \right| = \frac{1}{d\varphi(n)}.$$

Так как значение $\varphi(n)$ неизвестно, вместо $\frac{e}{\varphi(n)}$ можно использовать соотношение известных значений $\frac{e}{n}$ ввиду относительной близости чисел n и $\varphi(n)$ друг к другу:

$$\begin{aligned} |n - \varphi(n)| &= n - pq + p + q - 1 = \\ &= p + q - 1 < 2q + q - 1 < 3q < 3\sqrt{n}. \end{aligned}$$

Тогда при $k < d$ и $d < \frac{1}{3}\sqrt[4]{n}$ выполняется неравенство $3k < 3d < \sqrt[4]{n}$, откуда следует:

$$\left| \frac{e}{n} - \frac{k}{d} \right| = \left| \frac{\overbrace{ed - k\varphi(n)}^{=1} - kn + k\varphi(n)}{nd} \right| = \left| \frac{1 + k(\varphi(n) - n)}{nd} \right| < \frac{3k\sqrt{n}}{nd} \leq \frac{1}{3d^2} < \frac{1}{2d^2}.$$

Таким образом, подходящая дробь $\frac{k}{d}$ со знаменателем, близким к $\sqrt[4]{n}$, является хорошим приближением к $\frac{e}{n}$. Действуя таким образом, можно найти закрытый показатель d , если его длина достаточно мала.

Алгоритм 2.2. Атака Винера на криптосистему RSA

Вход. Число n , показатель e .

Выход. Закрытый показатель d .

1. Представить $\frac{e}{n}$ в виде обыкновенной непрерывной дроби $[0; a_1, a_2, \dots, a_l]$, где $l \approx \log_2 n$.

2. Для $i = 1, \dots, l$ выполнить следующие действия.

2.1. Вычислить i -ю подходящую дробь $\frac{P_i}{Q_i}$ к $\frac{e}{n}$:

$$\frac{P_i}{Q_i} = \frac{a_i P_{i-1} + P_{i-2}}{a_i Q_{i-1} + Q_{i-2}} \frac{P_i}{Q_i} = \frac{a_i P_{i-1} + P_{i-2}}{a_i Q_{i-1} + Q_{i-2}}, \text{ где } P_{-1} = 1, Q_{-1} = 0, P_0 = 0, Q_0 = 1.$$

2.2. Для произвольного сообщения $m \in \mathbb{Z}/n\mathbb{Z}$ проверить выполнение сравнения $(m^e)^{Q_i} \stackrel{?}{\equiv} m \pmod{n}$. Если сравнение выполняется, то положить $d \leftarrow Q_i$ и перейти на шаг 3.

3. Результат: d . ■

3. Случай специальных открытых показателей

Каждый i -й пользователь системы связи с шифрованием по схеме RSA должен иметь свое персональное число $n_i = p_i q_i$. Для ускорения процесса шифрования иногда используются малые открытые показатели e_i , причем они могут быть одинаковыми, например, 3 или 5. В этом случае, даже если числа n_i различны, можно выполнить бесключевое дешифрование сообщения. Предположим, что отправитель посылает одно и то же сообщение m трем получателям, открытые ключи которых равны (n_i, e_i) , причем $e_i = 3$ и $m < n_i$. Обозначим $x = m^3$. Тогда выполняются сравнения $c_i \equiv m^3 \pmod{n_i}$, $x \equiv c_i \pmod{n_i}$, и значение x как целое число может быть восстановлено по китайской теореме об остатках по известным c_i и взаимно простым модулям n_i . Для нахождения сообщения m нужно извлечь кубический корень из x в кольце целых чисел. Эта операция имеет полиномиальную сложность. Очевидно, то же справедливо и в случае других малых одинаковых открытых показателей.

Алгоритм 2.3. Бесключевое дешифрование широковещательного сообщения в случае малого общего показателя e

Вход. Открытые ключи нескольких пользователей: $(e, n_1), (e, n_2), \dots, (e, n_z)$, шифртексты широковещательного сообщения: $c_1, c_2, \dots, c_z$.

Выход. Сообщение m .

1. По известным шифртекстам $c_1, c_2, \dots, c_z$ и модулям $n_1, n_2, \dots, n_z$ по китайской теореме об остатках восстановить число $x = m^e$.

2. Вычислить $m = \sqrt[e]{x} \in \mathbb{Z}$.

3. Результат: m . ■

Еще одна атака на схему шифрования возможна, если мал порядок элемента e в группе $(\mathbb{Z}/\varphi(n)\mathbb{Z})^*$. Для нахождения сообщения m по шифртексту $c = m^e \pmod{n}$ необходимо выполнить последовательное зашифрование шифртекста: $c_1 = c^e \pmod{n}$, $c_2 \equiv c_1^e \pmod{n}$

и т. д. до получения шифртекста c , тогда предыдущее значение и будет открытым текстом.

Алгоритм 2.4. Бесключевое дешифрование сообщения в случае малого порядка элемента e в группе $(\mathbb{Z}/\varphi(n)\mathbb{Z})^*$

Вход. Составное число n , шифртекст c , открытый показатель e .

Выход. Открытый текст m .

1. Для $i = 1, 2, \dots$ выполнить следующие действия.

1.1. Вычислить $c_i \equiv c_{i-1}^e \equiv c_0^{e^i} \pmod{n}$, где $c_0 \leftarrow c$.

1.2. Проверить выполнение сравнения $c_i \stackrel{?}{\equiv} c \pmod{n}$. Если сравнение выполняется, то положить $m \leftarrow c_{i-1} \equiv c_0^{e^{ord(e)-1}} \pmod{n}$ и перейти на шаг 3.

2. Результат: m . ■

Отметим, что порядок элемента e в группе $(\mathbb{Z}/\varphi(n)\mathbb{Z})^*$ мал лишь в том случае, если он мал в обеих группах $\mathbb{F}_p^*$ и $\mathbb{F}_q^*$. Если этот порядок мал хотя бы в одной из указанных групп, то можно разложить число n следующим образом. Сначала сгенерировать случайное $a_0 \in (\mathbb{Z}/n\mathbb{Z})^*$. Затем для $i \geq 1$ вычислять последовательно значения $a_i \equiv a_{i-1}^e \pmod{n}$ и проверять с помощью алгоритма Евклида условие $\text{НОД}(a_i - a_0, n) > 1$. Если это условие будет выполнено, то указанный наибольший общий делитель является нетривиальным делителем числа n . Для исключения этой атаки числа p и q выбирают так, чтобы порядки групп $\mathbb{F}_p^*$ и $\mathbb{F}_q^*$ имели большие простые делители.

В случае малых открытых показателей можно не только осуществить бесключевое дешифрование, но и вскрыть закрытый показатель d . Для этого достаточно знать четверть двоичных разрядов одного из делителей числа n .

Контрольные вопросы

1. При каком условии возможна атака Винера?
2. Перечислите методы противодействия атаке Винера.
3. Почему числа p и q необходимо выбирать так, чтобы $p-1, q-1$ имели большие простые делители?

4. Предложите способы защиты от атаки бесключевого дешифрования широковещательных сообщений без отказа от использования общих малых закрытых показателей.

Порядок выполнения работы

1. Получить у преподавателя вариант задания.
2. Разработать программу **П-1**, которая реализует разложение составного числа на множители по известным показателям RSA и вычисляет закрытый ключ другого пользователя в случае общего модуля n . Программа **П-1** должна принимать на вход открытый и закрытый ключ RSA, а также открытый ключ RSA другого пользователя; на выходе – возвращать множители модуля RSA, закрытый ключ другого пользователя.
3. Разработать программу **П-2**, которая реализует атаку Винера на криптосистему RSA. Программа **П-2** должна принимать на вход открытый ключ RSA: число n и показатель e ; на выходе – возвращать закрытый показатель d .
4. Разработать программу **П-3**, которая, в соответствии с вариантом задания, реализует одну из следующих атак:
 - бесключевое дешифрование широковещательного сообщения в случае малого общего показателя e . Для восстановления числа m^e по китайской теореме об остатках и нахождения корня степени e допускается использовать готовые процедуры. Тело сообщения – файл, зашифрованный алгоритмом AES-256 в режиме CBC. При длине исходного файла, не кратной 16 байтам, открытый текст дополняется символами с кодом 0x03 так, чтобы длина сообщения (файла) перед зашифрованием была кратна 16 байтам. Изначальная длина доступна в элементе данных «Длина сообщения» заголовка. Удаление выравнивания не обязательно. Программа **П-3** должна принимать на вход открытые ключи нескольких пользователей, а также шифртексты широковещательного сообщения; на выходе – возвращать восстановленный открытый текст;
 - бесключевое дешифрование сообщения в случае малого порядка элемента e в группе $(\mathbb{Z}/\varphi(n)\mathbb{Z})^*$. Программа **П-3** должна при-

нимать на вход открытый ключ RSA, шифртекст; на выходе – возвращать восстановленный открытый текст.

5. Разработать программу **П-4**, которая осуществляет генерацию параметров криптосистемы RSA. Разработка процедур генерации случайных чисел и проверки числа на простоту не требуется. Генерация ключей должна обеспечивать выработку безопасных параметров криптосистемы с учетом произведенных атак. Программа **П-4** должна принимать на вход битовую длину модуля RSA; на выходе – возвращать параметры криптосистемы: n, e, d . Выполнить анализ сгенерированных параметров с помощью инструмента CcryptTool. Описание утилиты представлено в *Приложении Б*.

Содержание отчета

1. Формулировка задания.
2. Выполненная работа:
 - а) описание осуществленных атак;
 - б) параметры схемы RSA, используемые при атаках;
 - в) содержимое дешифрованных файлов и его тип, найденные значения закрытого ключа;
 - г) описание разработанной процедуры генерации параметров и ключей системы RSA;
 - д) пример сгенерированных ключей и параметров криптосистемы как последовательности модуля и экспоненты;
 - е) результаты тестирования сгенерированных параметров криптосистемы с помощью программы CcryptTool.
3. Ответы на контрольные вопросы.
4. Выводы по работе.
5. Листинги программ.

ПРОТОКОЛ ЭЛЕКТРОННОЙ ЦИФРОВОЙ ПОДПИСИ ГОСТ Р 34.10–2018

Цель работы – изучение протокола электронной цифровой подписи ГОСТ Р 34.10–2018, безопасность которого основана на задаче дискретного логарифмирования в группе точек эллиптической кривой.

Теоретические сведения

Эллиптические кривые над конечными полями в настоящее время представляют наиболее популярные структуры для построения криптографических алгоритмов. Это объясняется тем, что эллиптические кривые позволяют реализовать весь спектр криптографических протоколов с открытым ключом; задача дискретного логарифмирования на эллиптической кривой имеет экспоненциальную стойкость, которая практически не снижается во времени.

Действующий стандарт электронной цифровой подписи (ЭЦП) ГОСТ Р 34.10–2018, как и стандарт подписи США DSS, использует эллиптические кривые над конечными полями.

1. Эллиптические кривые в форме Вейерштрасса

Пусть $E(K)$ – эллиптическая кривая над полем K , заданная уравнением в форме Вейерштрасса

$$y^2 + a_1xy + a_3y = x^3 + a_2x^2 + a_4x + a_6,$$

где частные производные по x и по y полинома, задающего кривую, не обращаются в нуль одновременно ни в одной точке кривой даже при переходе к алгебраически замкнутому полю.

В криптографических алгоритмах на эллиптических кривых в качестве поля K чаще всего используется конечное поле $\mathbb{F}_p$. Если характеристика p отлична от 2 и 3, уравнение кривой можно записать в виде

$$y^2 = x^3 + Ax + B, \quad (1)$$

где $4A^3 + 27B^2 \neq 0 \pmod{p}$.

Для эллиптической кривой $E(\mathbb{F}_p)$ определен инвариант

$$j = 12^3 \cdot 4A^3(4A^3 + 27B^2)^{-1} \pmod{p}.$$

Точки эллиптической кривой образуют абелеву группу с геометрическим законом сложения, нулем группы является бесконечно удаленная точка P_∞ , точки (x, y) и $(x, -y)$ являются противоположными.

Для сложения точек $P_1 = (x_1, y_1)$ и $P_2 = (x_2, y_2)$ через них проводят секущую, которая пересекает кривую в третьей точке P_3 с координатами $(x_3, -y_3)$. Тогда точка (x_3, y_3) , противоположная к точке P_3 , по определению является суммой точек P_1 и P_2 . Закон сложения описывается следующими формулами. Если $P_1 \neq P_2$, то координаты точки $P_3 = P_1 + P_2 = 2P_1$ равны

$$x_3 = \lambda^2 - 2x_1 \pmod{p},$$

$$y_3 = \lambda(x_1 - x_3) - y_1 \pmod{p},$$

где угловой коэффициент равен $\lambda = (3x_1^2 + A)(2y_1)^{-1} \pmod{p}$.

Если $P_1 \neq \pm P_2$, то координаты точки $P_3 = P_1 + P_2$ равны

$$x_3 = \lambda^2 - x_1 - x_2 \pmod{p},$$

$$y_3 = \lambda(x_1 - x_3) - y_1 \pmod{p},$$

где угловой коэффициент равен $\lambda = (y_2 - y_1)(x_2 - x_1)^{-1} \pmod{p}$.

Формулы удвоения и сложения точек содержат операцию модульного обращения, которая является достаточно трудоемкой (выполняется с использованием расширенного алгоритма Евклида). Поэтому зачастую эллиптическую кривую записывают в проективной форме. Переход к проективной плоскости осуществляется заменой переменных: $x = XZ^{-1}, y = YZ^{-1}$.

Кривая (1) в нормальной форме Вейерштрасса в проективной плоскости задается уравнением

$$y^2Z = X^3 + AXZ^2 + BZ^3 \pmod{p} \quad (2)$$

с отношением эквивалентности

$$(X, Y, Z) \sim (uX, uY, uZ),$$

где $u \in \mathbb{F}_p^*$.

Любой класс эквивалентности троек $(X, Y, Z) \sim (uX, uY, uZ)$ с ненулевой Z -координатой содержит единственную тройку вида $(X, Y, 1)$. Переход от точки кривой в аффинной плоскости (к точке кривой в проективной плоскости (производится при помощи добавления к координатам третьей координаты $Z = 1$.

$$(x, y) \rightarrow (X = x, Y = y, Z = 1).$$

Обратный переход от (к (осуществляется с помощью «сокращения» координат

$$(X, Y, Z) = \left(\frac{X}{Z}, \frac{Y}{Z}, 1\right) \rightarrow (x = X, y = Y).$$

Для каждой точки P определена противоположная точка, проективные координаты которой удовлетворяют соотношению

$$-P = -(X, Y, Z) = (X, -Y, Z).$$

Для кривой (формулы удвоения в проективных координатах $(X_3, Y_3, Z_3) = 2(X_1, Y_1, Z_1)$ имеют вид:

$$\begin{aligned} X_3 &\equiv 2Y_1Z_1((3X_1^2 + AZ_1^2)^2 - 8X_1Y_1^2Z_1) \pmod{p}, \\ Y_3 &\equiv 4Y_1^2Z_1(3X_1(3X_1^2 + AZ_1^2) - 2Y_1^2Z_1) - (3X_1^2 + AZ_1^2)^3 \pmod{p}, \\ Z_3 &\equiv 8Y_1^3Z_1^3 \pmod{p}. \end{aligned}$$

Формулы сложения двух различных точек в проективных координатах $(X_3, Y_3, Z_3) = (X_1, Y_1, Z_1) + (X_2, Y_2, Z_2)$ для этой же кривой имеют вид:

$$\begin{aligned} X_3 &\equiv (X_2Z_1 - X_1Z_2)(Z_1Z_2(Y_2Z_1 - Y_1Z_2)^2 - \\ &\quad - (X_2Z_1 + X_1Z_2)(X_2Z_1 - X_1Z_2)^2) \pmod{p}, \\ Y_3 &\equiv (X_2Z_1 - X_1Z_2)^2(Y_2Z_1(X_2Z_1 + 2X_1Z_2) - Y_1Z_2(X_1Z_2 + 2X_2Z_1)) - \\ &\quad - Z_1Z_2(Y_2Z_1 - Y_1Z_2)^3 \pmod{p}, \\ Z_3 &\equiv Z_1Z_2(X_2Z_1 - X_1Z_2)^3 \pmod{p}. \end{aligned}$$

2. Умножение точки на число

Операция умножения точки P эллиптической кривой на целое число k представляет собой k -кратное применение групповой операции к точке P : $kP = \underbrace{P + P + \dots + P}_{k \text{ раз}}$, при этом $-kP = -(kP) = k(-P)$.

Наиболее простым методом умножения точки на число является бинарное умножение. При использовании бинарного умножения чис-

ло, на которое умножается точка, представляется в двоичной системе счисления.

Алгоритм 6.1. Бинарное умножение точки на число

Вход: Точка эллиптической кривой P . Целое положительное число k .

Выход: Произведение kP .

1. Если $k = 0$, то результат: P_∞ – бесконечно удаленная точка.
2. Представить число k в двоичной системе счисления: $k = (k_{t-1} \dots k_1 k_0)_2$.

3. Положить $Q \leftarrow P_\infty$.

4. Для $i = t - 1, \dots, 0$ выполнить:

- 4.1. $Q \leftarrow 2Q$ (удвоение точки).

- 4.2. Если $k_i \neq 0$, положить $Q \leftarrow Q + P$ (сложение точек).

5. Результат Q . ■

Алгоритм бинарного умножения точки на число во многом схож с алгоритмом модульного возведения в степень.

3. Стандарт подписи ГОСТ Р 34.10–2018

Стандарт ГОСТ Р 34.10–2018 подготовлен на основе стандарта ГОСТ Р 34.10–2012. Криптографическая стойкость схемы цифровой подписи определяется сложностью решения задачи дискретного логарифмирования в группе точек эллиптической кривой, а также стойкостью используемой в протоколе хэш-функции ГОСТ Р 34.11–2018.

Задача дискретного логарифмирования в группе точек эллиптической кривой формулируется следующим образом: для известной точки $P \in E(\mathbb{F}_p)$ (образующей подгруппы группы точек эллиптической кривой E) и данной точки $Q \in \langle P \rangle$ найти показатель d такой, что $Q = dP$.

Протокол подписи стандарта ГОСТ Р 34.10–2018 предусматривает использование эллиптической кривой E в форме (1) над простым полем $\mathbb{F}_p$ ($p > 3$). Эллиптическая кривая задается инвариантом или коэффициентами A, B , вычисление которых возможно по известному значению инварианта:

$$\begin{cases} A = 3k \pmod{p}, \\ B = 2k \pmod{p}, \end{cases}$$

где $k = j(1728 - j)^{-1} \pmod{p}$, $j \neq 0$, $j \neq 1728$.

Параметры цифровой подписи:

- 1) p – простое число (модуль эллиптической кривой);
- 2) эллиптическая кривая E (коэффициенты A, B или $j(E)$);
- 3) m – порядок группы точек эллиптической кривой $\#E(\mathbb{F}_p)$;
- 4) q – простой делитель порядка группы точек m : $m = n \cdot q$, $n \in \mathbb{Z}, n > 1$. Длина числа q определяется в соответствии со стандартом на хэш-функцию ГОСТ Р 34.11–2018: $2^{254} < q < 2^{256}$ или $2^{508} < q < 2^{512}$;

- 5) точка $P \in E(\mathbb{F}_p)$, $P \neq P_\infty$, $qP = P_\infty$, $\# \langle P \rangle = q$.

Дополнительные требования:

- 1) $p^t \neq 1 \pmod{q}, \forall t = 1, \dots, D$, где $D = 31$, если $2^{254} < q < 2^{256}$; $D = 131$, если $2^{508} < q < 2^{512}$;
- 2) $m \neq p$;
- 3) $j(E) \neq 0, 1728$.

В основу протокола электронной цифровой подписи ГОСТ Р 34.10–2018 положен протокол Эль-Гамала. Пусть M – подписываемое сообщение, h – хэш-функция по ГОСТ Р 34.11–2018, $\{E(\mathbb{F}_p), P, q\}$ – параметры цифровой подписи; число d – ключ подписи ($0 < d < q$), $Q = dP$ – ключ проверки.

Протокол 6.1. Схема подписи ГОСТ Р 34.10–2018

Вход отправителя. Закрытый ключ d , параметры подписи, сообщение M .

Вход получателя. Открытый ключ Q , параметры подписи.

Результат. Формирование и проверка подписи для сообщения M .

Для формирования подписи для сообщения M отправитель выполняет следующие действия.

1. Вычисляет хэш-образ сообщения M : $\bar{h} \leftarrow h(M)$.
2. Вычисляет число $\alpha \in \mathbb{Z}$, двоичным представлением которого является вектор $\bar{h}$. Находит $e \equiv \alpha \pmod{q}$, при $e = 0$ полагает $e \leftarrow 1$.

3. Выбывает случайный показатель $k \in \mathbb{Z}$, $0 < k < q$.
4. Вычисляет точку $C \leftarrow kP$, $C = (x_C, y_C)$, $r \equiv x_C \pmod{q}$. При $r = 0$ возвращается на шаг 3.
5. Вычисляет $s \leftarrow (rd + ke) \pmod{q}$, при $s = 0$ возвращается на шаг 3.
6. Вычисляет двоичные векторы, соответствующие числам r, s : $r \rightarrow \bar{r}, s \rightarrow \bar{s}$. Цифровая подпись имеет вид $\xi = (\bar{r} || \bar{s})$.

Для проверки подписи получатель выполняет следующие действия.

1. Переводит двоичные векторы в числа: $\bar{r} \rightarrow r, \bar{s} \rightarrow s$, где $r, s \in \mathbb{Z}$. Проверяет выполнение неравенств $0 < r < q$, $0 < s < q$. Если хотя бы одно из них не выполняется, то подпись неверна.
2. Вычисляет хэш-образ сообщения: $\bar{h} \leftarrow h(M)$.
3. Вычисляет число $\alpha \in \mathbb{Z}$, двоичным представлением которого является вектор $\bar{h}$. Находит $e \equiv \alpha \pmod{q}$, при $e = 0$ полагает $e \leftarrow 1$.
4. Вычисляет $v \equiv e^{-1} \pmod{q}$.
5. Вычисляет $z_1 \equiv sv \pmod{q}$, $z_2 \equiv -rv \pmod{q}$.
6. Находит точку $C = z_1P + z_2Q$, $C = (x_C, y_C)$. $R \equiv x_C \pmod{q}$.
7. Проверяет выполнение равенства $R \stackrel{?}{=} r$. Если оно выполнено, то подпись принимается, в противном случае – подпись неверна.

■

Контрольные вопросы

1. Перечислите преимущества криптосистем на эллиптических кривых по сравнению с другими криптосистемами.
2. Почему в стандарте ГОСТ Р 34.10–2018 введено требование $\#E(\mathbb{F}_p) \neq p$?
3. Если нарушитель имеет возможность обращаться хэш-функцию, как он может подделать сообщение и подпись?
4. Почему случайный показатель k не должен повторяться в течение времени жизни ключа?

Порядок выполнения работы

Получить у преподавателя вариант задания и разработать программу **П-1**, реализующую протокол подписи согласно ГОСТ Р 34.10–2018. Описание формата файла подписи представлено в *Приложении Г*. Параметры криптосистемы соответствуют варианту задания и приведены в *Приложении Д*. Программа должна поддерживать функции формирования и проверки подписи, а также допускать возможность использования различных ключей.

Содержание отчета

1. Формулировка задания.
2. Выполненная работа:
 - а) использованные параметры криптосистемы;
 - б) пример файла, используемого для подписи. При значительном размере файла – первые 80–100 байт;
 - в) хэш-образ сообщения (файла);
 - г) содержимое файла подписи в шестнадцатеричном виде (при значительном размере файла – первые 80–100 байт).
3. Ответы на контрольные вопросы.
4. Выводы по работе, где сделать предположения о возможных уязвимостях разработанного программного обеспечения и о путях их устранения.
5. Листинг программы.

ОПИСАНИЕ ФОРМАТА ASN.1

Любые данные в ASN.1 представляются в виде последовательности «Тэг – Длина – Значение». При этом «Тэг» определяет тип данных и имеет длину 1 байт, а «Длина» задает размер следующего поля «Значение». В ASN.1 предусмотрены как базовые типы, приведенные в таблице А.1, так и сложные (составные) типы данных, приведенные в таблице А.2.

Таблица А.1 – Базовые типы данных в ASN.1

Код	Тип	Комментарий
0x02	INTEGER	Целое число
0x0C	UTF_STRING	Строка в кодировке UTF-8
0x04	BYTE_STRING	Последовательность байтов
...	...	...

Таблица А.2 – Сложные (составные) типы данных в ASN.1

Код	Тип	Комментарий
0x30	SEQUENCE	Последовательность
0x31	SET	Строка в кодировке UTF-8
...	...	...

В значениях базовых типов записываются элементарные данные. В значениях составных типов могут присутствовать только другие поля. Причем, в SEQUENCE – поля любых типов, однако обычно в известном порядке, а в SET – только одного и того же типа, а порядок следования элементов не имеет значения.

1. Сохранение длины полей

Сохраняемые значения не имеют ограничений по длине (теоретическое ограничение – 256^{127} байт). Поле «Длина» имеет определенный формат:

1) если длина меньше 0x80 (128) байтов, тогда в поле «Длина» записывается число в бинарном виде без изменений;

2) если длина больше или равна 128 байтам, тогда в первом байте в младших семи битах сохраняется длина следующих байтов поля «Длина», а старший бит выставляется в единицу. Следующие байты, количество которых определено первым, задают непосредственно длину поля данных. Порядок байтов – MSB.

Разбор поля «Длина» производится следующим образом. Сначала проверяется старший бит старшего байта $((b \& 0x80) == 0)$, & – побитовое И). Если он нулевой, длина менее 128 байт и сохранена в первом байте, то общая длина поля «Длина» – 1 байт.

Если старший байт единичный $((b \& 0x80) != 0)$, тогда применен префикс поля «Длина». Информация сохранена в последующих k байтах, причем $k = b \& 0x7F$ (младшие 7 битов). Считываются следующие k байтов и преобразуются в число. Обычно k не более 2–3.

2. Примеры представления ключей RSA

Для сохранения структуры предназначен тип SEQUENCE. Он сохраняет последовательность любых базовых и составных типов, в частности может включать в себя другие последовательности.

Обычно ключ RSA сохраняют как последовательность модуля и экспоненты, каждый из которых является целым числом.

Рассмотрим пример. Пусть сохраняется ключ RSA

$$e = 3,$$

$$n = (11317E789C45CCCC7436C384D4354945)_{16}.$$

Первый элемент последовательности – модуль n . В самом числе n старший бит старшего байта равен 0, поэтому дописывание нулем не требуется. Представление числа n имеет длину 16 байтов, которая может сохраняться без префикса. Поэтому ASN.1-представление числа n будет иметь вид

0x02 10 11317E789C45CCCC7436C384D4354945,

а именно:

	TAG	LEN	Value	
	0x02	0x10	11317E4945	

Представление числа 3 также без дописывания нулями и без префикса длины: 02013

	TAG	LEN	Value
	0x02	0x01	0x03

Последовательность имеет тэг 0x30. Длина первого представления составляет 18 байт, второго – 3 байта. Общая длина данных последовательности – 21 байт = 0x15 – также может быть записана без префикса длины.

TAG	LEN	TAG	LEN	Value	TAG	LEN	Value
0x30	0x15	Элемент 1			Элемент 2		
		0x02	0x10	11317E....4945	0x02	0x01	0x03

Полученный общий результат будет равен

3015 0210 11317E789C45CCCC7436C384D4354945 020103.

К достоинствам описанного представления стоит отнести универсальность, стандартизованность и заложенные возможности по определению ошибок. К недостаткам – необходимость привлечения дополнительных вычислительных мощностей для обработки данных перед отправкой или после получения, избыточность данных.

Описанный формат хранения ключей RSA является достаточно распространенным при сохранении данных ключа в сертификатах, а также в запросах и ответах аппаратных модулей безопасности, работающих с RSA.

3. Формат зашифрованного файла

Файл состоит из заголовка и зашифрованной при помощи симметричного алгоритма шифрования последовательности данных.

Заголовок представляет собой ASN.1-последовательность ключей и данных о файле. Первый символ файла – 0x30, далее длина всего заголовка и данные. Содержимое заголовка приведено в таблице А.3.

Множество ключей представляет собой ASN.1-множество, элементами которого являются последовательности с данными об использованных ключах. Каждая последовательность представляет собой открытый ключ и соответствующий ему шифртекст. Описание последовательностей приведено в таблице А.4. Дополнительные данные приведены в таблице А.5.

Таблица А.3 – Содержимое заголовка

Поле	Тэг	Комментарии	Значение элемента
Множество ключей	31	Данные заголовка об использованных ключах	0x31 <i>len_keys</i> <i>key1_seq</i> , ... <i>keyN_seq</i>
Открытые данные о файле (сообщении)	30	Дополнительные данные о файле (сообщении). Опционально	0x30 0x00

Таблица А.4 – Описание последовательностей

Поле	Тэг	Комментарии	Значение элемента
Идентификатор алгоритма шифрования	04	0x0001 – RSA	0x04 0x02 0x00 0x00
Строковый идентификатор ключа, псевдоним	0C	Опционально – строковый идентификатор ключа. Если не указан, следует использовать пустую строку 0x0C, 0x00	0x0C <i>len_i</i> UTF8(id)
Открытый ключ	30	Для RSA – последовательность модуля и экспоненты	0x30, <i>len_s</i> , 0x02, <i>len_n</i> , <i>n</i> , 0x02, <i>len_e</i> , <i>e</i>
Параметры алгоритма	30	Для RSA не используется, общий параметр <i>n</i> передается внутри открытого ключа	0x30 0x00
Шифртекст алгоритма с открытым ключом (зашифрованный промежуток)	30	Для RSA – последовательность, состоящая из одного числа <i>c</i> .	0x30, <i>len_s</i> , 0x02, <i>len_c</i> , <i>c</i>

Поле	Тэг	Комментарии	Значение элемента
жуточный ключ симметричного алгоритма)		Если ключ не указан, можно использовать последовательность нулевой длины: 0x30, 0x00	

где,

n – модуль RSA длины len_n ;

e – открытая экспонента RSA длины len_e ;

c – шифртекст алгоритма RSA длины len_c ;

len_s – длины последовательностей.

Таблица А.5 – Дополнительные данные

Поле	Тэг	Комментарии	Значение элемента
Идентификатор алгоритма шифрования	04	Идентификатор симметричного алгоритма 0x0121 – DES в режиме простой замены 0x0132 – Triple-DES в режиме простой замены 0x0133 – Triple-DES в режиме CBC 0x1081 – AES в режиме простой замены 0x1082 – AES в режиме CBC	0x04 <i>len_fclfile_content_len</i>

Поле	Тэг	Комментарии	Значение элемента
Длина сообщения	02	Опционально – длина передаваемого сообщения (файла). Если длина не указана, следует использовать пустую строку 0x02, 0x00. Используется для выравнивания	0x0C <i>len_fclfile_content_len</i>

Заголовок с данными:

303F31323030040200010C04746573743015021011317E789C45CCCC74
36C384D43549450201033000300B02090A0B0C0A0B0C0A0B0C300904
0201210203133870

разбирается как ASN.1-структура следующего вида:

Start Tag Len

0 30 63 : SEQUENCE { – заголовок
2 31 50 : SET { – множество ключей
4 30 48 : SEQUENCE { – первый ключ
6 04 2 : OCTET STRING – идентификатор алгоритма
(RSA)
: 00 01
10 0C 4 : UTF8String 'test'
16 30 21 : SEQUENCE { – значение открытого ключа
18 02 16 : INTEGER – модуль, число p
: 11 31 7E 78 9C 45 CC CC 74 36 C3 84 D4 35 49
45
36 02 1 : INTEGER 3 – открытая экспонента, число e
: }
39 30 0 : SEQUENCE {} – параметры криптосистемы
41 30 11 : SEQUENCE { – шифртекст
43 02 9 : INTEGER – число c

```

        :      0A 0B 0C 0A 0B 0C 0A 0B 0C
        :      }
        :      }
        :      }
54 30  9 : SEQUENCE {
56 04  2 : OCTET STRING
        :      01 32 – идентификатор симметричного алгорит-
ма, Triple-DES
60 02  3 : INTEGER 1259632 – длина файла
        :      }
        :      }

```

4. Формат файла подписи

Формат файла подписи совпадает с форматом файла сообщения, однако файл подписи не содержит в себе тела сообщения. Дополнительные данные о файле также могут быть пропущены, поскольку для подписи не несут никакой важной информации.

Пример файла подписи:

```

3041313D303B040200060C08746573745369676E3015021011317E
789C45CCCC7436C384D435494502010330003012021009F505B178C43
16B0BDAE8C0162254F53000

```

```

0 30  65 : SEQUENCE {– заголовок
2 31  61 : SET { – множество ключей (обычно только 1)
4 30  59 : SEQUENCE { – ключ и подпись
6 04  2 : OCTET STRING
        :      00 06 – идентификатор алгоритма (RSA-SHA1)
10 0C  8 : UTF8String 'testSign' – строковый идентификатор
ключа
20 30  21 : SEQUENCE {
22 02  16 : INTEGER – модуль, число n
        :      11 31 7E 78 9C 45 CC CC 74 36 C3 84 D4 35 49
45
40 02  1 : INTEGER 3 – открытая экспонента, число e
        :      }
43 30  0 : SEQUENCE {}

```

```

45 30 18 : SEQUENCE { – подпись сообщения
47 02 16 : INTEGER – число s
          : 09 F5 05 B1 78 C4 31 6B 0B DA E8 C0 16 22 54
F5
          : }
          : }
          : }
65 30 0 : SEQUENCE {} – дополнительных данных нет
          : }

```

Наряду с алгоритмом подписи RSA-SHA1 допустимо использовать алгоритм RSA-SHA256, который использует в качестве хэш-функции алгоритм SHA-256. Идентификатор алгоритма – 0x0040.

РАБОТА С УТИЛИТОЙ CRYPTOOOL

CrypTool (www.cryptool.org) – открытый образовательный проект, направленный на изучение и анализ различных криптографических алгоритмов. CrypTool поддерживает визуализацию современных криптографических примитивов, генерацию параметров, методы криптоанализа. В рамках проекта существует несколько утилит, которые отличаются функционалом:

- CrypTool 1 – версия, предназначенная для операционных систем семейства Windows и включающая в себя большинство современных криптографических примитивов, а также визуализацию теоретико-числовых алгоритмов;
- версия CrypTool 2 основана на концепции визуального программирования и позволяет пользователю комбинировать различные алгоритмы и функции;
- JCrypTool доступен для платформ на базе Windows, Linux, MacOS. Помимо классических шифров включает в себя некоторые алгоритмы постквантовой криптографии.

Версия CrypTool 1 поддерживает:

- классические шифры: шифр Цезаря, Виженера, Атбаш, Playfair и др.;
- современные симметричные шифры: IDEA, RC2, RC4, RC6, DES, 3DES, AES и др.;
- шифрование и подпись RSA, криптографию на эллиптических кривых (в том числе комбинированные схемы RSA-AES, ECC-AES);
- хэш-функции: MD2, MD5, SHA-1, SHA-256, SHA-512, RIPEMD-160 и др.
- генерацию псевдослучайных чисел и их тестирование (тест серий, покер-тест и др.), генерацию ключей;
- base64- и UU-кодирование и др.;
- методы анализа файлов (энтропийный, автокорреляционный анализ, построение гистограмм и др.);

- методы криптоанализа: разложение на множители модуля RSA, атака на RSA с использованием решеток, генерация коллизий хэш-функции и др.;
- образовательная игра «Number Shark»;
- визуализация операции сложения точек на эллиптической кривой, принципов работы алгоритмов AES, DES, шифровальной машины Энигма, учебные материалы по теории чисел и др.

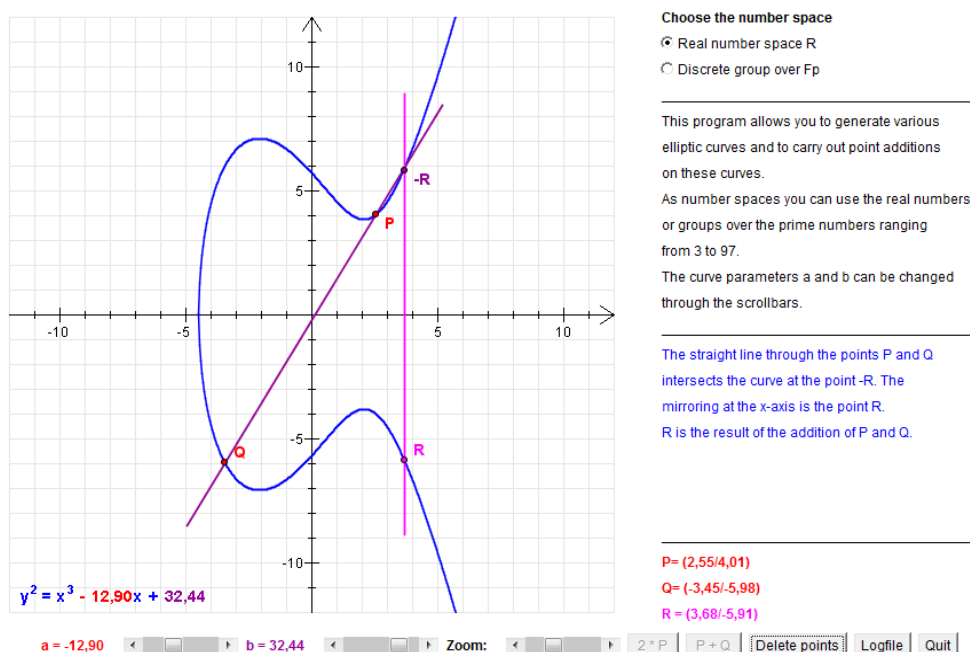


Рисунок Б.1 – Визуализация закона сложения точек на эллиптической кривой в CrypTool 1

Утилита CrypTool 1 предоставляет возможность анализа параметров криптосистемы RSA. Для этого в пункте «Asymmetric Encryption» раздела «Analysis» выбрать «Factorization of a Number», ввести значение модуля n и запустить процедуру тестирования, нажав «Complete factorization into primes».

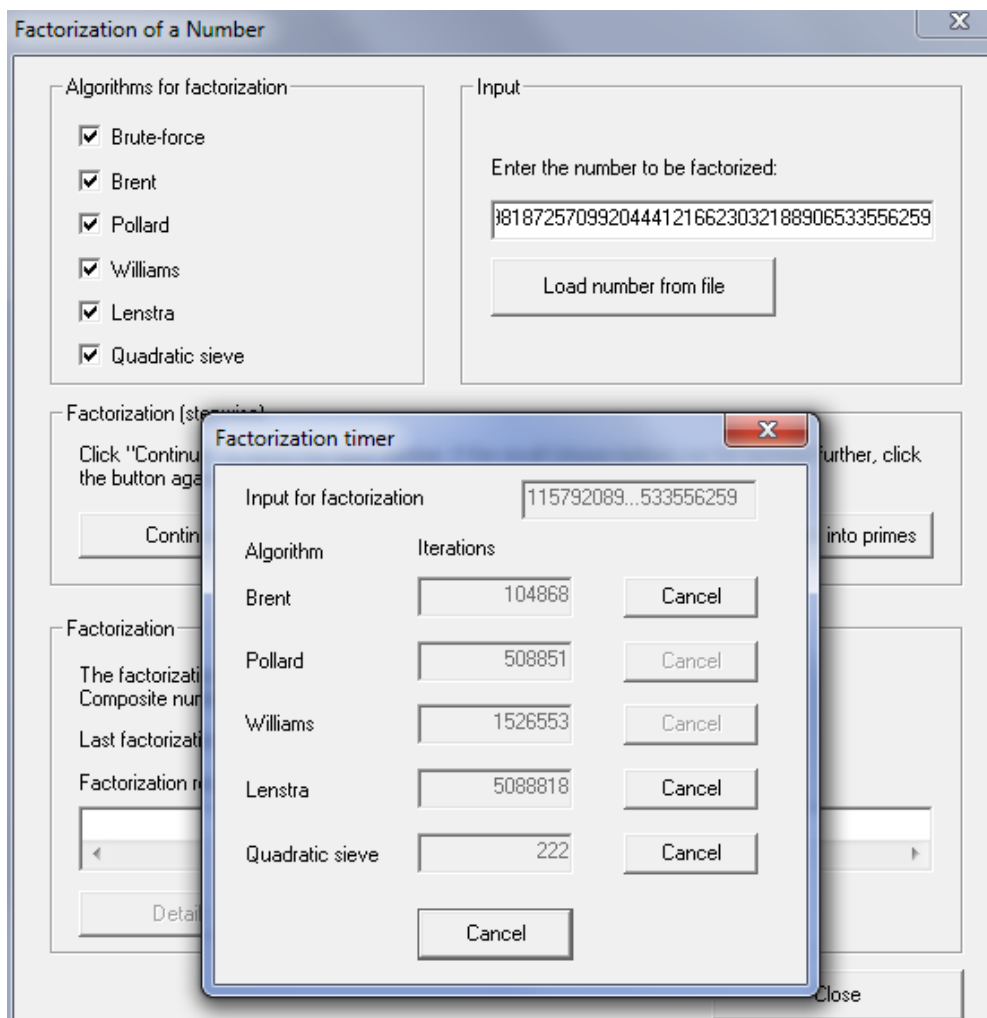


Рисунок Б.2 – Разложение на множители в CrypTool 1

Помимо алгоритмов разложения на множители CrypTool 1 реализует атаку на криптосистему RSA с использованием решеток (раздел «Analysis/Asymmetric Encryption/Lattice-Based Attack on RSA»), которая эффективна при следующих предположениях:

- в случае частично известного открытого текста;
- закрытый показатель d слишком мал по сравнению с n ;
- частично известен один из множителей числа n .

Attack on Small Secret Exponents (according to Bloemer / May)

Description
 This attack factors an RSA modulus N if the secret key d is too small compared to N .
 The number $\delta = \log(d)/\log(N)$ is called "size of d ". The attack is feasible for $\delta < 0.290$.

☐ To apply examples from the literature, first enter the public key (N, e) .
 Then enter the estimated value of δ . Alternatively, you can directly enter d to calculate δ .
☒ To generate random values, enter the desired δ and bit length of N .
 Then click on "Generate random RSA key".

Then click "Start".

Step 1: Enter key parameters and key

Bit length of N : δ :

N :

e :

d :

Step 2: Enter attack parameters for the lattice base reduction

m : Determines the size of the lattice to reduce and the maximum size of δ . Should be at least 4.

t : Optimally calculated as a function of m .

Lattice dimension: Size of the lattice to reduce. Impacts the running time significantly.

Maximum δ : Maximum size of δ for large N ($N > 1000$ Bit).

Step 3: Start attack

Building lattice:

Reducing lattice: Reductions:

Calculating resultant: Resultants:

Overall time:

Found factorization:

p : q :

Рисунок Б.3 – Атака на RSA в случае небольшой длины закрытого показателя d в CrypTool 1

Таким образом, утилита CrypTool предоставляет широкий набор инструментов для изучения принципа действия различных криптосистем, визуализации атак, генерации и тестирования параметров.

ASN.1- ФОРМАТ СООБЩЕНИЙ ДЛЯ КРИПТОСИСТЕМ НА ОСНОВЕ ЗАДАЧИ ДИСКРЕТНОГО ЛОГАРИФМИРОВАНИЯ

1. Варианты заданий

Вариант	Протокол/ схема	Идентификатор
1	Протокол шифрования Эль-Гамала	0x80010201, $f(m, b^y) = mb^y \pmod{p}$
2		0x80010202, $f(m, b^y) = m + b^y \pmod{p}$
3		0x80010203, $f(m, b^y) = m - b^y \pmod{p}$
4	Протокол Диффи–Хеллмана	0x0021
5		
6	Протокол Месси–Омуры	0x80070200
7		
8	Схема подписи Эль-Гамала	0x80060200
9		
10		

2. Реализация протокола Эль-Гамала шифрования с открытым ключом

Пусть $G = \mathbb{F}_p^*$, $M = G$. Формат файла для передачи является обобщением формата, используемого для лабораторной работы № 1 и описанного в *Приложении А*. Однако в файле могут быть представлены различные типы ключей.

Шифртекстом является последовательность чисел

```
ElGamalEncryptedMessage ::= SEQUENCE {
    ay – INTEGER,
    c – INTEGER
}
```

Открытым ключом является последовательность

```
ElGamalPublicKey ::= SEQUENCE {
    b – INTEGER – открытый ключ
}
```

Параметры протокола содержат параметры группы $G = \mathbb{F}_p^*$: характеристику поля и образующую:

```
ElGamalParameters ::= SEQUENCE {
    prime INTEGER, – простое число  $p$ 
    generator INTEGER – образующая  $a$ 
}
```

3. Реализация протокола Диффи–Хеллмана

Параметры протокола содержат параметры группы $G = \mathbb{F}_p^*$: характеристику поля и образующую:

```
DHParameters ::= SEQUENCE {
    prime INTEGER, – простое число  $p$ 
    generator INTEGER – образующая  $a$ 
}
```

Сообщения протокола представляются в виде последовательности из одного числа

```
DHMessage ::= SEQUENCE {
     $a^x$  или  $a^{xy}$  – INTEGER,
}
```

Открытый ключ не задействуется и представляет собой пустую последовательность.

Сторона 1. Клиент пытается установить TCP-соединение с сервером.

Клиент генерирует случайный закрытый показатель x и передает на сервер значение a^x . Также передаются параметры системы: образующая a , число p . Таким образом, на сервер передается три параметра (a^x, a, p) . Формат сообщения:

```
SEQUENCE { – заголовок
    SET { – множество ключей, 1 задействован
        SEQUENCE { – первый ключ
```

OCTET STRING – идентификатор алгоритма (протокол Диффи–Хеллмана)

00 21

UTF8String 'dh' – может не задействоваться или использоваться для идентификации будущего установленного ключа

SEQUENCE { } – значение открытого ключа, не используется

SEQUENCE {– параметры криптосистемы

INTEGER – простое число p

PP PP ... PP

INTEGER – образующая a

AA AA ... AA

}

SEQUENCE {– шифртекст, показатель a^x

INTEGER – число c

CC CC ... CC

}

}

}

SEQUENCE {} – параметры файла, не используются

}

Сторона 2. Сервер ведет ожидание входящих подключений.

При входящем подключении сервер принимает тройку (a^x, a, p) , генерирует случайный показатель y и передает клиенту значение a^y . Также вычисляется значение a^{xy} . Клиенту передается сообщение следующего формата:

SEQUENCE {– заголовок

SET {– множество ключей, 1 задействован

SEQUENCE {– первый ключ

OCTET STRING – идентификатор алгоритма (протокол Диффи–Хеллмана)

00 21

UTF8String 'dh' – может не задействоваться или использоваться для идентификации будущего установленного ключа

SEQUENCE { } – значение открытого ключа, не используются

SEQUENCE { } – параметры криптосистемы

SEQUENCE { – шифртекст

INTEGER – значение a^{xy}

CC CC ... CC

}

}

}

SEQUENCE {} – параметры файла, не используются

}

Параметры при передаче ответа сервером могут быть пустыми или скопированы из запроса.

Сторона 1. Клиент принимает a^y , вычисляет a^{xy} .

Из установленного между клиентом и сервером закрытого ключа a^{xy} получается ключ AES: $K = a^{xy} \bmod 2^{256}$.

Далее клиент и сервер могут обмениваться сообщениями зашифрованными алгоритмом AES.

Клиент должен быть способен передать короткое текстовое сообщение на сервер. Сообщение имеет следующий формат.

SEQUENCE { – заголовок

SET { } – множество ключей, не задействовано

SEQUENCE { – параметры файла (сообщения)

OCTET STRING

10 82 – идентификатор симметричного алгоритма, AES

INTEGER XXXX – длина файла до выравнивания

}

}

Формат заголовка совпадает с форматом в лабораторной работе № 1, однако секция с открытыми ключами не задействуется.

Тело сообщения дописывается после заголовка. Если длина сообщения не кратна 16 байтам, то открытый текст дополняется символами с кодом 0x03 так, чтобы длина сообщения (файла) перед зашифрованием была кратна 16 байтам.

При передаче необходимо указывать длину открытого текста до зашифрования. Длина содержимого, которое необходимо считать из входного потока ТСП-соединения, кратно длине блока симметричного шифра и определяется как наименьшее число, которое превосходит длину сообщения до выравнивания.

Сторона 2. Сервер принимает, расшифровывает и выводит на экран принятое сообщение.

4. Реализация протокола бесключевого шифрования Мессии–Омуры

Сторона 1. Клиент (отправитель) пытается установить ТСП-соединение с сервером.

После успешной установки соединения клиент производит передачу сообщения, вычисляет $t \leftarrow f(m)$ для сообщения m . Если t является единичным элементом e группы G , то результат: сообщение зашифрованию не подлежит.

Отправитель также генерирует случайный показатель a , обратимый по модулю r , вычисляет элемент t^a группы G и передает на сервер t^a получателю.

На сервер передаются параметры (однократно, в первом сообщении диалога)

```
MasseyOmuraParameters ::= SEQUENCE {  
  prime INTEGER, – простое число  $p$   
   $r$  INTEGER, – порядок  $r$  группы  
}
```

и шифртекст

```
MasseyOmuraMessage1 ::= SEQUENCE {  
   $t^a$  – INTEGER – первый шифртекст,  $t^a$   
}
```

Формат сообщения:

SEQUENCE { – заголовок

SET { – множество ключей, один задействован

SEQUENCE { – первый «ключ»

OCTET STRING – идентификатор алгоритма (протокол Мессии–Омуры)

80 07 02 00

UTF8String 'mo' – может не задействоваться или использоваться для идентификации будущего переданного сообщения

SEQUENCE { } – значение открытого ключа, не используется

SEQUENCE { – параметры криптосистемы

INTEGER – простое число p

PP PP ... PP

INTEGER – порядок r группы

RR RR ... RR

}

} – параметры криптосистемы

SEQUENCE { – шифртекст

INTEGER – показатель t^a

XX XX ... XX

}

}

}

SEQUENCE { } – параметры файла, не используются

}

Сторона 2. Сервер ведет ожидание входящих подключений.

При соединении клиента принимает информацию, вырабатывает случайный показатель b , обратимый по модулю r , вычисляет значение $(t^a)^b = t^{ab}$ и посылает его отправителю.

Клиенту передается шифртекст (параметры в ответ не включаются). Формат сообщения:

SEQUENCE { – заголовок

SET { – множество ключей, один задействован

SEQUENCE { – первый «ключ»

OCTET STRING – идентификатор алгоритма (протокол Мессиа–Омуры)

80 07 02 00

UTF8String 'mo' – может не задействоваться или использоваться для идентификации будущего переданного сообщения

SEQUENCE { } – значение открытого ключа, не используется

ся

SEQUENCE { } – параметры криптосистемы, не используются

SEQUENCE {– шифртекст
INTEGER – значение t^{ab}
XX XX ... XX
}
}
}

SEQUENCE { } – параметры файла, не используются
}

Сторона 1. Клиент (отправитель) возводит полученный шифртекст в степень a^{-1} : $(t^{ab})^{a^{-1}} = t^b$ и посылает значение t^b получателю.

Содержимое сообщения аналогично предыдущему, однако сообщение дополняется результатом шифрования файла при помощи алгоритма AES:

SEQUENCE {– заголовок
SET {– множество ключей, один задействован
SEQUENCE {– первый «ключ»
OCTET STRING – идентификатор алгоритма (протокол Мессинг–Омуры)

80 07 02 00

UTF8String 'mo' – может не задействоваться или использоваться для идентификации будущего переданного сообщения

SEQUENCE { } – значение открытого ключа, не используется

SEQUENCE { } – параметры криптосистемы, не используются

SEQUENCE {– шифртекст
INTEGER – показатель t^b
XX XX ... XX
}
}
}
SEQUENCE {

OCTET STRING

10 82– идентификатор симметричного алгоритма, AES

INTEGER XXXX – длина файла до выравнивания

}– параметры файла, не используются

}

Ключ $k = m \bmod 2^{256}$ алгоритма AES. Выравнивание сообщения и определение длины исходного сообщения выполняются аналогично вариантам 4, 5.

Сторона 2. Сервер возводит полученный шифртекст в степень b^{-1} : $(t^b)^{b^{-1}} = t$ и вычисляет $m \leftarrow f^{-1}(t)$. По известному m вычисляется ключ AES и переданное сообщение.

5. Реализация схемы подписи Эль-Гамала

Формат файла для передачи является обобщением формата, используемого для лабораторной работы № 1 и описанного в *Приложении А*. Однако в файле могут быть представлены различные типы ключей.

Подписью сообщения является последовательность чисел:

ElGamalSign ::= SEQUENCE {

w – INTEGER,

s – INTEGER

}

Ключом является последовательность:

ElGamalSignPublicKey ::= SEQUENCE {

b – INTEGER – открытый ключ

}

Параметры протокола содержат параметры группы $G = \mathbb{F}_p^*$: характеристику поля, порядок группы и образующую:

ElGamalSignParameters ::= SEQUENCE {

prime INTEGER, – число p

r INTEGER, – порядок группы r

generator INTEGER – образующая a

}

Параметры файла (сообщения) не используются, следует использовать последовательность нулевой длины. Дополнение открытого текста перед вычислением хэш-образа не производится.

ASN.1-ФОРМАТ ФАЙЛА ПОДПИСИ ГОСТ Р 34.10–2018

Общий формат файла подписи:

SEQUENCE {– заголовок

SET {– множество ключей, 1 задействован

SEQUENCE {– первый «ключ»

OCTET STRING – идентификатор алгоритма (протокол подписи ГОСТ)

80 06 07 00

UTF8String 'gostSignKey' – может не задействоваться

SEQUENCE {– значение открытого ключа

INTEGER - x -координата точки Q

QX QX ... QX

INTEGER - y -координата точки Q

QY QY ... QY

}

SEQUENCE {– параметры криптосистемы

SEQUENCE {– параметры поля

INTEGER – простое число p

PP PP ... PP

}

SEQUENCE {– параметры кривой

INTEGER – коэффициент A уравнения кривой

AA AA ... AA

INTEGER – коэффициент B уравнения кривой

BB BB ... BB

}

SEQUENCE {– образующая группы точек кривой

INTEGER – x -координата образующей точки P

PX PX ... PX

INTEGER – y -координата образующей точки P

PY PY ... PY

}

INTEGER – порядок группы q

RR RR ... RR

} – параметры криптосистемы

SEQUENCE { – подпись сообщения

INTEGER – число r

XR XR ... XR

INTEGER – число s

SS SS ... SS

}

}

SEQUENCE {} – параметры файла, не используются

}

Все значения приведены в десятичной системе счисления

Вариант 1

p = 57896044620753384133869303843568937902752767818974600847634902975134129543643
r = 28948022310376692066934651921784468951377218528270520403696863131129758387393

a = 1
b = 52259530098387149819562511889780651425271270942919542722038553712464420235875
xP = 14539175448068301073584752148116082765715462525899666138074034449285211025933
yP = 8328801466633898282311029798556417767141491055036399348346324804478619400451

Вариант 2

p = 57896044628890729911196718984933305846544100325488685311213142875135838763683
r = 28948022314445364955598359492466652923270809441897180344196391207096541510137

a = 1
b = 51597193811365919768190236681066502033803499635094541650610225403695076439048
xP = 21371456824977467041033238171905463424508399897529674896678501178686263573482
yP = 52962982709744467108853563358242537068648343861092009194618855518747612108192

Вариант 3

p = 57896044625414088412406986721186632159605151965036429316594800028484330862739
r = 28948022312707044206203493360593316079803694388568974763893400879284219004579

a = -1
b = 53520245325288251180656443226770638951803337703360722011463033447827147086694
xP = 36066034950041118412594006918367965339490267219250288222432003968962962331642
yP = 54906983586985298119491343295734802658016371303757622466870297979342757624191

Вариант 4

p = 57896044625259982827082014024491516445703215213774687456785671200359045162371
r = 28948022312629991413541007012245758222850495633896873081323396140811733708403

a = -1
b = 53956679838042162451108292176931772631109916272820066466458395232513766926866
xP = 12933162268009944794066590054824622037560826430730236852169234350278155715869
yP = 18786030474197088418858017573086015439132081546303111294023901101650919011383

Вариант 5

p = 57896044630612021680684936114742422271145183870487080309667128995208157569947
r = 28948022315306010840342468057371211135571302038761442251594012761075345324491

a = 1
b = 19750513962881385028059495396984460236743646692126413053976069443380491067343
xP = 43490682822985073571091311123441225129011272278165566160439297012894969619553
yP = 53273700124912449490307054424387372532482586733448415163119878489682918137700

Вариант 6

p = 57896044622894643241131754937450315750132642216230685504884320870273678881443
r = 28948022311447321620565877468725157875067316353637126186229732812867492750347

a = 1
b = 41431894589448105498289586872587560387979247722721848579560344157562082667257
xP = 54672615043105947691210796380713598019547553171137275980323095812145568854782
yP = 42098178416750523198643432544018510845496542305814546233883323764837032783338

Вариант 7

p = 57896044622894643241131754937450315750132642216230685504884320870273678881443
r = 28948022311447321620565877468725157875067316353637126186229732812867492750347

a = 1
b = 41431894589448105498289586872587560387979247722721848579560344157562082667257
xP = 54672615043105947691210796380713598019547553171137275980323095812145568854782
yP = 42098178416750523198643432544018510845496542305814546233883323764837032783338

Вариант 8

p = 57896044628958718631213028275518411328476149599789770738757218840632915517411
r = 28948022314479359315606514137759205664236832023231628035871193493020981068937

a = -1
b = 44516423948019661825420813927965592341675839270849860441861020678502941837466
xP = 2323576058601956664720966708045726308916627824741707729836708887517232685058
yP = 20772302011053991390127435262297715010367018383131467831609444907978987653753

Вариант 9

p = 57896044623304043104299010705220227593411047292536827792690781397890347747339
r = 28948022311652021552149505352610113796704505182029833954354922263763927570003

a = -1
b = 43710617685176640081115791799072828565464240578102647674571004997445073590733
xP = 2375898023066818922259876468809957958853252818454947955735283254927910319797
yP = 41494802599112005706154126874294477637447794334726831230592311725680992538390

Вариант 10

p = 57896044623332830704464930175758866532580959320654190378215212919875855638347
r = 28948022311666415352232465087879433266289459622068172692541485718987902249339

a = 1
b = 3580094891504076339485469867608308463096783610651696491327180352870738800622
xP = 34716160583222611645030789937385729759908872067425542316787922148347820731789
yP = 5898821206414759138201363717802568499793444816690169368839685124167835466530